# PRODUCTION DATABASE MIGRATION AND VERSION 2 ROLLOUT

*Engineering Case Study*

| Field | Description |
|---|---|
| Author | Venkata Lakshmi Parimala Pasupuleti |
| Domain | Enterprise Affordable Housing Software |
| System Type | Production database and application-version migration |
| Primary Focus | Data correctness, controlled rollout, validation, rollback readiness, evidence-based incident response |
| Status | Production experience documented from reported project participation and ownership |
| Document Version | 1.0 |

**Scope note**

This document is grounded in the reported Version 2 rollout: a production database migration, a concern that the migration was incorrect, comparison of database dumps, collaboration with the database team, phased rollout, timestamp confusion, and evidence that supported the migration result. Exact table names, row counts, migration framework, and internal commands were not provided. Where the document describes broader controls or modernization patterns, they are presented as engineering guidance rather than claims about the original implementation.

# Contents

1. Executive Summary
2. Business and Release Context
3. Problem Statement and System Goals
4. Ownership and Collaboration Model
5. Requirements
6. Migration Architecture Overview
7. Pre-Migration Discovery and Impact Analysis
8. Migration Script Design and Rehearsal
9. Data Validation and Reconciliation
10. Deployment and Phased Rollout
11. Rollback and Recovery Strategy
12. Production Investigation Timeline
13. Evidence-Based Root-Cause Analysis
14. Database Transactions, Locking, and Concurrency
15. Observability and Auditability
16. Failure Modes and Risk Controls
17. Design Decisions and Trade-offs
18. Lessons Learned
19. Modernization Direction
Appendix A. Migration Runbook
Appendix B. Validation Checklist
Appendix C. Glossary

# 1. Executive Summary

A production database migration is not merely a schema change. It is a coordinated transformation of data, application assumptions, release sequencing, and operational responsibility. The Version 2 rollout described in this case study required the application and database to move together while preserving business continuity and customer data integrity.

The most important engineering challenge was establishing confidence in the migration outcome. During the release, a concern was raised that the migration had been performed incorrectly. Instead of treating the concern as proof of failure or reacting with an immediate destructive rollback, the team collected database dumps, reviewed timestamps and execution evidence, and worked with the database team to reconstruct what had occurred. The investigation ultimately showed that the migration evidence supported correctness and that timestamp interpretation had contributed to the confusion.

This experience reinforced several senior engineering principles: migration scripts must be repeatable and reviewable; validation must be defined before release; backups and rollback paths must be available; evidence must be preserved throughout execution; and high-pressure production decisions should be driven by data rather than authority, intuition, or fear.

The case study also documents a phased rollout model. A phased approach reduces blast radius, creates checkpoints for validation, and allows the team to stop before a problem becomes universal. It does not remove risk, but it makes risk observable and manageable.

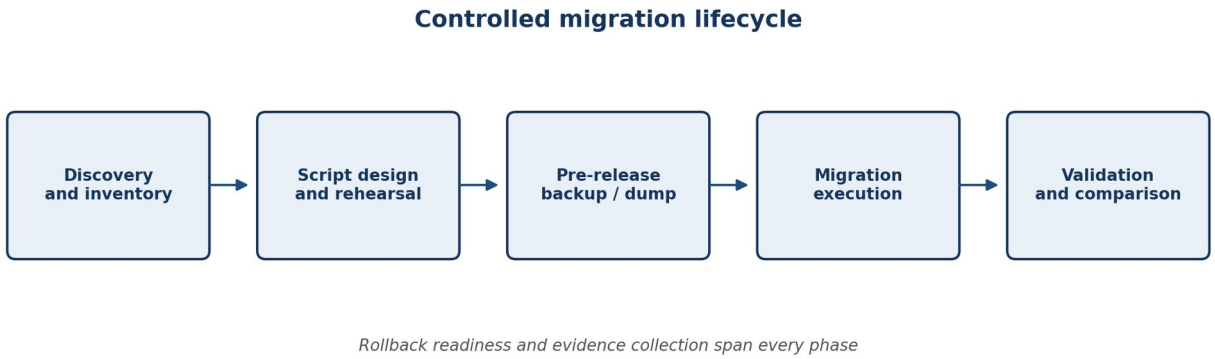



*Figure 1. A controlled database-migration lifecycle with continuous rollback readiness and evidence collection.*

# 2. Business and Release Context

The migration occurred as part of a Version 2 rollout. A major version introduces more risk than an isolated patch because application code, database structure, stored behavior, and existing data may all need to remain compatible during the transition. Even when the code path is correct, release order can create a temporary mismatch between old application assumptions and new database state.

In an Affordable Housing platform, database correctness has direct business impact. Data supports eligibility, rent calculations, compliance workflows, resident certifications, customer documents, and historical auditability. A migration defect can therefore appear far away from the script that introduced it. The first visible symptom may be a customer workflow rather than a database exception.

The release required collaboration among application engineering, database engineering, QA, management, and production operations. Each group viewed the release through a different lens: application behavior, database safety, functional correctness, delivery risk, and operational continuity. Successful execution depended on combining those perspectives.

## 2.1 Why a Version 2 migration is high risk

- Existing production records must satisfy the assumptions of the new version.
- New schema objects or constraints may change write behavior and query plans.
- Old and new application versions may coexist during a phased deployment.
- Historical records may contain exceptions that were tolerated by the previous version.
- A migration can complete technically while still producing semantically incorrect data.
- Rollback may require reversing both database and application changes in a coordinated order.

## 2.2 Primary business objective

Move production safely to the Version 2 data model and application behavior without losing data, corrupting relationships, or interrupting critical customer workflows.

# 3. Problem Statement and System Goals

The core problem was to transform the production database into the state expected by Version 2 while maintaining a defensible path to recovery. The migration needed to account for existing data, application compatibility, release sequencing, and post-deployment verification.

A technically successful script was not sufficient. The team needed proof that the resulting data represented the intended business state. The release therefore required both execution controls and validation controls.

## 3.1 System goals

- Preserve all required customer and historical data.
- Apply schema and data transformations in a controlled, reviewable sequence.

- Keep migration execution deterministic and repeatable wherever possible.
- Provide before-and-after evidence for data reconciliation.
- Limit blast radius through staged or phased deployment.
- Enable rapid recovery if validation fails.
- Maintain a clear operational record of what ran, when it ran, and what changed.

## 3.2 Non-goals

- Using the migration as an opportunity for unrelated refactoring.
- Introducing new infrastructure without a release requirement.
- Accepting unverified assumptions about production data quality.
- Changing business meaning while presenting the work as a purely technical conversion.

# 4. Ownership and Collaboration Model

The application engineer owned the application-side understanding of the migration: what Version 2 expected, how business workflows used the affected data, and how to determine whether post-migration behavior remained correct. The database team contributed database-level expertise, production access patterns, dump comparison, and independent review of execution evidence.

This shared model matters because a database migration crosses organizational boundaries. Application engineers understand semantic meaning, while database engineers understand execution mechanics, locking, recovery, and infrastructure behavior. Neither perspective is sufficient by itself.

During the production concern, the engineer did not rely solely on the initial statement that the migration was wrong. The investigation moved toward observable evidence: database dumps, timestamps, migration results, and comparison with expected state. That behavior is a central example of senior-level ownership under pressure.

| Role | Primary responsibility during migration |
|---|---|
| Application engineer | Business semantics, application compatibility, functional validation, migration intent, production triage |
| Database engineer / DBA | Database execution, backups or dumps, locking and performance review, independent data comparison |
| QA | Representative workflow validation, regression coverage, defect confirmation |
| Release owner / manager | Go/no-go coordination, communication, sequencing, risk acceptance |
| Support / operations | Customer-impact signals, issue intake, escalation context |

**Ownership principle**
A senior engineer does not need to personally execute every database operation to own the outcome. Ownership means understanding the intent, validating the result, coordinating specialists, and staying accountable through production stabilization.

# 5. Requirements

## 5.1 Functional requirements

- Create or alter the database structures required by Version 2.
- Transform existing production records into the new representation.
- Preserve required relationships, identifiers, and historical meaning.
- Allow Version 2 application paths to read and write successfully after rollout.
- Produce a deterministic result when executed against the same starting state.
- Support validation of both migrated records and unaffected records.

## 5.2 Non-functional requirements

| Quality attribute | Requirement |
|---|---|
| Correctness | No silent loss, duplication, or semantic corruption of production data. |
| Recoverability | A defined rollback or restore path must exist before execution. |
| Auditability | Execution time, script version, operator, and validation outcome should be traceable. |
| Availability | Customer disruption should be minimized and communicated. |
| Performance | Migration duration and locking should remain within an acceptable release window. |
| Repeatability | Rehearsal and production execution should use the same reviewed migration artifacts. |
| Compatibility | Release order must prevent unsupported application/database combinations. |

# 6. Migration Architecture Overview

The migration architecture can be understood as four cooperating planes: release orchestration, database transformation, application deployment, and validation. The release succeeds only when all four reach a compatible state.

The database transformation plane applies schema and data changes. The application plane introduces Version 2 behavior. The orchestration plane controls order and checkpoints. The validation plane independently determines whether the transformed system is safe to continue.

| Plane | Responsibilities | Primary evidence |
|---|---|---|
| Release orchestration | Sequence, maintenance window, checkpoints, go/no-go decisions | Run log, timestamps, approvals |
| Database transformation | DDL, DML, data conversion, constraints, indexes | Script output, row counts, database dump |
| Application deployment | Version 2 code, configuration, compatibility handling | Deployment logs, health checks |
| Validation | Structural, data-quality, business, and workflow verification | Comparisons, queries, QA results |

## 6.1 Compatibility sequence

The safest order depends on whether the database changes are backward compatible. A common controlled sequence is: deploy additive database changes, migrate or backfill data, validate, deploy Version 2 application code, then remove deprecated structures only after the system has stabilized. When the original migration was not designed this way, the principle remains useful as a modernization direction rather than a claim about the historical implementation.

**Critical release invariant**
At every checkpoint, the currently running application version must be compatible with the database state that exists at that moment.

# 7. Pre-Migration Discovery and Impact Analysis

Before a production migration, the team must understand not only the target schema but also the actual shape of production data. Development and test environments rarely contain every historical exception, partially completed workflow, or customer-specific configuration found in production.

Impact analysis maps each affected object to application queries, writes, reports, stored procedures, integrations, and customer workflows. This creates the dependency view needed for release sequencing and validation.

## 7.1 Discovery activities

- Inventory tables, columns, indexes, constraints, triggers, procedures, and views affected by Version 2.
- Identify all application reads and writes that depend on the old representation.
- Profile production data for nulls, duplicates, invalid references, and uncommon legacy states.
- Determine whether large tables require batched updates to control locks and transaction log growth.
- Identify downstream exports, reports, documents, and jobs that consume the migrated data.
- Define representative customer scenarios for business validation.

## 7.2 Dependency register

| Dependency type | Examples of questions |
|---|---|
| Application code | Which endpoints, jobs, or modules read or write the affected fields? |
| Database logic | Do stored procedures, triggers, or views assume the prior schema? |
| Operational jobs | Will scheduled tasks run during the migration window? |
| Reporting | Do reports depend on column names, formats, or historical mappings? |
| Integrations | Will external consumers receive a changed payload or identifier? |
| Customer configuration | Can a setting alter how migrated data is interpreted? |

# 8. Migration Script Design and Rehearsal

A migration script should be treated as production software. It requires version control, peer review, deterministic ordering, clear preconditions, explicit failure behavior, and a rehearsal against production-like data.

Rehearsal is not only a performance test. It validates assumptions about historical data, execution order, transaction size, constraint behavior, and script rerun safety. A migration that works against a clean test database may fail against years of production history.

## 8.1 Script design principles

- Fail fast when required preconditions are not satisfied.
- Use explicit transactions where atomicity is realistic and beneficial.
- For large transformations, use controlled batches and durable progress markers.
- Make data transformations idempotent or clearly detect prior execution.
- Separate irreversible cleanup from the initial compatibility migration.
- Write validation queries alongside the migration, not after it.
- Record a migration identifier and execution timestamp in an audit table or release log.

## 8.2 Rehearsal outputs

| Output | Purpose |
|---|---|
| Execution duration | Confirms the release window is realistic. |
| Lock profile | Identifies blocking or long-running transactions. |
| Before/after counts | Confirms expected coverage and data movement. |
| Exception list | Documents records that cannot be converted automatically. |
| Rerun result | Confirms safe behavior after partial execution or restart. |
| Application smoke test | Shows the target version can use the migrated state. |

# 9. Data Validation and Reconciliation

Validation must answer more than “Did the script finish?” A completed command proves execution, not correctness. A reliable migration validates structure, volume, data quality, business meaning, and the final application workflow.

Before-and-after evidence is especially important when production concerns arise. Database dumps and timestamped results create an objective record that can be compared independently of memory or interpretation.

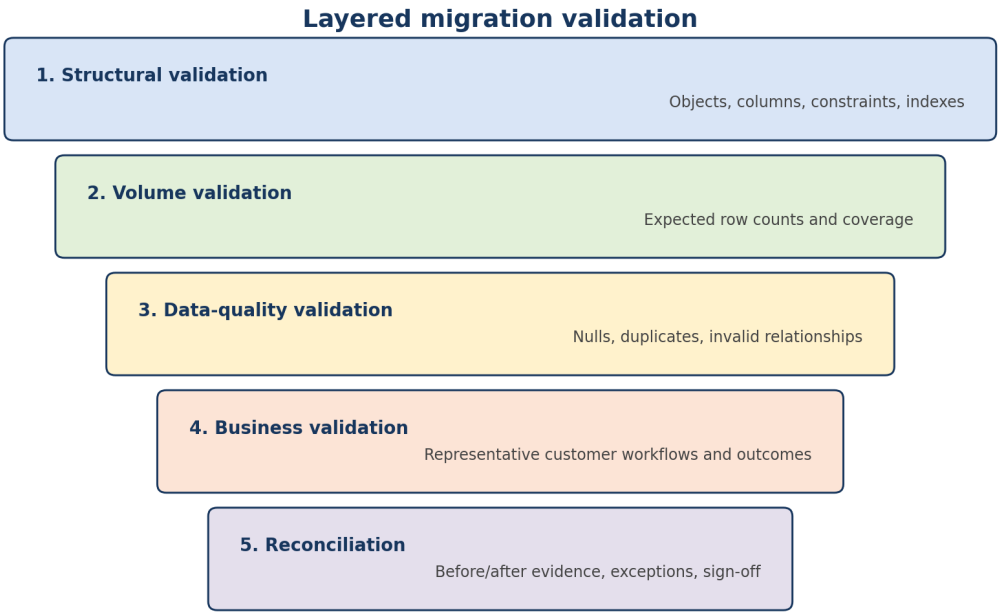



*Figure 2. Layered validation moves from database structure to business outcomes and reconciliation.*

## 9.1 Structural validation

- Required tables, columns, data types, indexes, and constraints exist.
- Deprecated structures remain available if the release requires backward compatibility.
- Database objects compile and dependent views or procedures are valid.

## 9.2 Volume and quality validation

- Expected source and target row counts reconcile, accounting for intentional splits or consolidations.
- No unexpected duplicates were introduced.
- Foreign-key and logical relationships remain intact.
- Null and default-value behavior matches the target design.
- Exception records are identified and assigned an explicit disposition.

## 9.3 Business validation

- Representative Version 2 workflows produce the expected results.
- Historical records remain readable and meaningful.
- Critical downstream modules continue to operate.
- Customer-facing output is checked where the migration affects documents or calculations.

# 10. Deployment and Phased Rollout

The rollout was phased rather than treated as a single irreversible switch. A phased rollout creates bounded checkpoints where the team can validate the system before increasing exposure. The exact phases depend on the platform, but the governing principle is to reduce blast radius and preserve the ability to stop.

Phasing also improves diagnosis. When only a controlled subset is exposed, engineers can compare migrated and non-migrated behavior, correlate issues with release boundaries, and avoid interpreting unrelated customer behavior as migration impact.

## 10.1 Illustrative rollout phases

1. Freeze or control conflicting writes and capture a pre-migration backup or database dump.
2. Apply reviewed database changes to the controlled production scope.
3. Run structural and data reconciliation checks.
4. Deploy or enable Version 2 for the initial scope.
5. Execute application smoke tests and representative business workflows.
6. Expand to the next scope only after explicit validation and sign-off.
7. Complete stabilization monitoring before irreversible cleanup.

## 10.2 Go/no-go checkpoints

| Checkpoint | Continue when | Stop or rollback when |
|---|---|---|
| Pre-execution | Backup verified; scripts and owners ready | Backup unavailable; prerequisites fail |
| Post-schema | Objects and dependencies validate | Missing/invalid objects or blocking behavior |
| Post-data | Counts and relationships reconcile | Unexpected loss, duplication, or orphaning |
| Post-application | Health checks and smoke tests pass | Version mismatch or critical workflow failure |
| Expansion | Initial scope remains stable | Customer impact or unexplained anomalies |

# 11. Rollback and Recovery Strategy

Rollback planning must be completed before migration execution. It is not enough to say that a backup exists; the team must know what would trigger rollback, who can authorize it, how long restoration will take, and how application state will be aligned afterward.

Not every migration should be reversed through a down script. For complex data transformations, restoring a known-good snapshot may be safer than attempting to reconstruct prior values. In other cases, a forward correction may be lower risk than rollback. The choice depends on data reversibility, release exposure, and customer impact.

## 11.1 Recovery options

| Option | Best suited for | Trade-off |
|---|---|---|
| Transaction rollback | Small atomic migrations before commit | Not feasible for long or multi-stage releases |
| Down migration | Reversible schema changes with reliable inverse logic | Inverse data transformations may lose information |
| Snapshot / dump restore | Severe corruption detected before substantial new writes | Requires downtime and loses post-snapshot changes |
| Forward correction | Localized, understood defect after release | Requires high confidence in diagnosis |
| Feature disable / app rollback | Database remains compatible with prior application | Depends on backward-compatible schema design |

**Rollback decision principle**

Rollback is not automatically the safest response. The team should compare the risk of remaining in the current state, restoring prior state, and applying a forward correction using current evidence and customer impact.

# 12. Production Investigation Timeline

After the migration, a concern was raised that the migration was wrong. This immediately increased release pressure because a database migration can affect many downstream workflows. The investigation required separating an assertion from evidence.

Database dumps were collected and compared, timestamps were reviewed, and the database team participated in the analysis. The timeline and data state were reconstructed rather than inferred. The investigation ultimately supported the migration result and identified timestamp interpretation as a source of confusion.

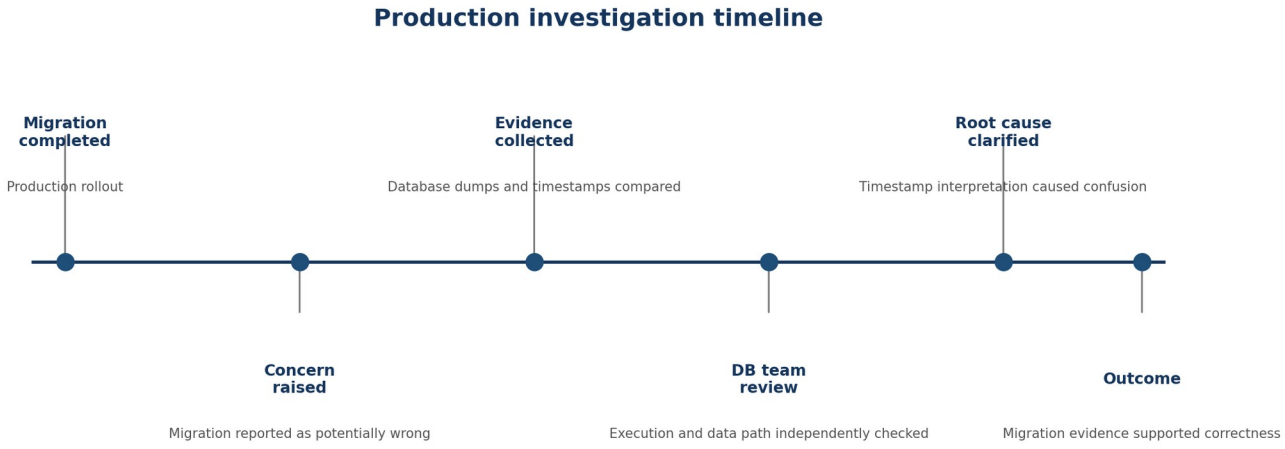



*Figure 3. Evidence-based reconstruction of the production migration concern.*

## 12.1 What made the incident difficult

- The initial concern came during a high-risk production release, when teams are naturally cautious.
- Timestamps can reflect different events: file creation, dump generation, script start, transaction commit, replication, or display timezone.
- A database dump is valuable evidence only when its capture point and source are clearly understood.
- Pressure to act quickly can encourage conclusions before the evidence is complete.
- Multiple teams may use different terminology for the same release event.

# 13. Evidence-Based Root-Cause Analysis

The investigation followed a principle that has become central to the engineer's production practice: validate claims using observable evidence. When a manager or teammate states that a migration is wrong, the statement is an important signal, but the next step is to determine what result differs from expectation and how that difference can be measured.

The database dumps provided snapshots that could be compared. Timestamps helped reconstruct sequence, but they also required careful interpretation. Working with the database team added an independent technical review and reduced the risk of application-side bias.

## 13.1 Evidence hierarchy

| Evidence | Question it answers | Caution |
|---|---|---|
| Migration script/version | What logic was intended to run? | Does not prove which version actually ran |
| Execution log | What commands ran and when? | Clock and timezone must be understood |
| Database dump | What state existed at a capture point? | Capture source and timing must be known |
| Row-level comparison | Which records differ from expectation? | Must distinguish expected transformation from defect |
| Application behavior | Can Version 2 use the result? | One workflow does not validate the whole dataset |
| DB team review | Does database evidence support the sequence? | Still requires shared definitions and artifacts |

## 13.2 Investigation method

**1.** Define the specific claim: which data, object, or workflow is believed to be wrong.

**2.** Identify the expected state and the source of that expectation.

**3.** Collect immutable or reproducible evidence from before and after the migration.

**4.** Normalize timestamps and verify timezone, source server, and transaction boundaries.

**5.** Compare records using business keys and transformation rules, not only raw row order.

**6.** Ask an independent database engineer to review the sequence and results.

**7.** Document the conclusion and the evidence supporting it.

# 14. Database Transactions, Locking, and Concurrency

Although exact transaction settings were not provided, every production migration must reason about atomicity, lock duration, concurrent writes, and recovery. A migration can be logically correct yet operationally dangerous if it blocks critical workloads or reads an inconsistent data set.

Large transactions provide strong atomicity but can increase lock duration, transaction-log growth, and rollback time. Small batches reduce operational pressure but require durable progress tracking and careful idempotency.

## 14.1 Transaction design considerations

- Choose transaction boundaries based on the smallest unit that must remain consistent.
- Prevent concurrent application writes when they can invalidate migration assumptions.
- Use explicit ordering to reduce deadlock risk.
- Estimate lock duration and test against production-like volume.
- Persist batch progress only after the associated data commit succeeds.
- Do not declare completion until all batches and reconciliation checks finish.

## 14.2 Compatibility with live traffic

When zero-downtime execution is required, additive schema changes and dual-read or dual-write techniques can maintain compatibility. These patterns add complexity and should be introduced only when the availability requirement justifies them. For a controlled maintenance window, temporarily restricting writes may be safer and simpler.

# 15. Observability and Auditability

Migration observability is different from normal application monitoring. The team needs evidence at the level of release steps, data coverage, exception counts, and validation outcomes. Standard CPU and error-rate dashboards are useful but cannot prove semantic correctness.

Timestamp confusion in the production investigation demonstrates why audit records need explicit meaning. A timestamp should be labeled as script start, transaction commit, dump capture, validation completion, or deployment activation rather than presented as a generic “time.”

| Signal | Recommended content |
|---|---|
| Migration audit record | Migration ID, version/hash, environment, operator, start/end, status |
| Progress metric | Total expected, processed, succeeded, skipped, failed |
| Validation record | Query/check name, expected result, actual result, timestamp, reviewer |
| Exception ledger | Business key, reason, remediation owner, final |

| Signal | Recommended content |
|---|---|
| | disposition |
| Release timeline | Database step, application step, feature activation, checkpoint decisions |
| Customer signal | Support issues or workflow errors correlated to rollout scope |

## 15.1 Minimum post-release dashboard

- Application error and latency trends for Version 2 workflows.
- Database lock waits, slow queries, and resource saturation.
- Counts of records in expected migrated states.
- Background jobs or downstream processes affected by the schema change.
- Customer support issues tagged to the release scope.
- Outstanding migration exceptions and reconciliation status.

# 16. Failure Modes and Risk Controls

| Failure mode | Potential impact | Control |
|---|---|---|
| Script partially executes | Mixed old/new data representation | Transactions, batch checkpoints, idempotent rerun logic |
| Wrong script version runs | Unexpected schema or transformation | Artifact hash, release manifest, peer verification |
| Concurrent writes change source data | Inconsistent conversion | Maintenance window, write controls, repeatable snapshot |
| Timestamp misinterpreted | Incorrect incident conclusion | UTC normalization and event-specific labels |
| Row counts match but semantics are wrong | Silent business defect | Business-key and workflow validation |
| Rollback path is untested | Extended outage during failure | Restore rehearsal and decision runbook |
| Old application reaches new schema | Runtime failure or incorrect writes | Compatibility matrix and ordered deployment |
| Large update creates blocking | Customer-facing degradation | Batching, lock testing, execution monitoring |
| Exceptions are ignored | Incomplete customer migration | Exception ledger and explicit sign-off |

**Risk posture**

The goal is not to prove that failure is impossible. The goal is to make failure detectable, bounded, diagnosable, and recoverable.

# 17. Design Decisions and Trade-offs

| Decision | Rationale and trade-off |
|---|---|
| Phased rollout instead of a single switch | Reduced blast radius and created validation checkpoints. The trade-off was a longer release process and the need to reason about temporary mixed states. |
| Evidence collection before destructive action | Protected against unnecessary rollback and enabled a defensible conclusion. The trade-off was spending time on investigation while release pressure remained high. |
| Database-team partnership | Combined application semantics with database operational expertise. The trade-off was coordination overhead and the need for shared terminology. |
| Database dumps for comparison | Provided durable before/after evidence. The trade-off was storage, capture time, and the need to interpret snapshot timing correctly. |
| Explicit validation beyond script completion | Protected business meaning, not only technical execution. The trade-off was additional release preparation and runtime. |
| Rollback readiness without automatic rollback | Allowed the team to choose the least risky recovery path. The trade-off was requiring judgment under pressure rather than following a simplistic rule. |

# 18. Lessons Learned

The most important lesson was that production decisions must be evidence based. A confident statement from any participant, including a manager, is not a substitute for database state, execution logs, or reproducible comparisons. Respectful technical disagreement is sometimes necessary to protect the system.

The incident also demonstrated that timestamps are data with semantics. Without timezone, source, and event meaning, a timestamp can create more confusion than clarity. Release artifacts should use a shared time standard and explicitly name each event.

Phased rollout proved valuable because it reduced exposure and made validation possible at intermediate points. The approach requires discipline: each phase must have explicit acceptance criteria rather than simply waiting for the absence of alerts.

Finally, migration ownership extends beyond writing SQL. It includes understanding the business model, coordinating specialists, planning recovery, validating production outcomes, and documenting the result so future teams do not repeat the same uncertainty.

### 18.1 Principles carried forward

- Prepare rollback and validation plans before the release window begins.
- Preserve before-and-after evidence for every high-risk data transformation.
- Define timestamps, timezones, and release events consistently.
- Use phased rollout to create real checkpoints, not ceremonial stages.
- Escalate uncertainty early, but do not convert uncertainty into an unsupported conclusion.
- Close incidents with documented evidence and improvements to the runbook.

## 19. Modernization Direction

The original production experience can be strengthened with a modern migration control plane. This section describes a target approach, not a claim about the historical implementation.

A modern design would store each migration as a versioned artifact with checksum, preconditions, execution history, and validation queries. The deployment pipeline would block incompatible application releases, run rehearsals against sanitized production-like data, and require automated reconciliation before expansion.

For large migrations, an online expand-migrate-contract strategy could separate additive schema changes, backfill, application cutover, and final cleanup. Progress and exceptions would be visible through an operational dashboard, while feature flags would control exposure to Version 2 behavior.

Automated controls should not remove human judgment. High-risk migrations still need an accountable release owner, explicit acceptance criteria, and the authority to pause when evidence is incomplete.

### 19.1 Target capabilities

- Versioned migration framework with checksums and execution history.
- Automated schema-drift and compatibility checks in CI/CD.
- Production-like rehearsal using masked or synthetic data at realistic volume.
- Automated before/after reconciliation and exception reports.
- UTC release timeline generated from deployment, database, and feature systems.
- Feature-flagged expansion with cohort-level health metrics.
- Central migration dashboard with owner, progress, failures, and sign-off.

# Appendix A. Migration Runbook

| Phase | Runbook action |
|---|---|
| Preparation | Confirm scope, script version, owners, communication channel, maintenance window, backup, and rollback decision maker. |
| Pre-check | Validate prerequisites, production data profile, available storage, replication health, scheduled jobs, and active writes. |
| Evidence capture | Record current schema, source counts, representative records, database dump identifier, and UTC timestamp. |
| Execution | Run approved artifact, capture logs, monitor locks/resources, and stop on defined failure conditions. |
| Validation | Run structural, volume, quality, business, and workflow checks; record results. |
| Application rollout | Deploy or enable Version 2 according to compatibility plan. |
| Expansion | Increase rollout scope only after explicit go/no-go approval. |
| Stabilization | Monitor system and customer signals; close exceptions and document the final outcome. |

# Appendix B. Validation Checklist

- ☐ Approved migration artifact and checksum confirmed.
- ☐ Pre-migration backup or database dump verified and restorable.
- ☐ All prerequisites and expected schema versions confirmed.
- ☐ Expected source and target row-count formulas documented.
- ☐ Null, duplicate, orphan, and exception checks prepared.
- ☐ Representative business records selected before execution.
- ☐ Application/database compatibility sequence reviewed.
- ☐ UTC timeline and event labels agreed by all release participants.
- ☐ Post-migration schema and object validation passed.
- ☐ Data reconciliation and exception disposition completed.
- ☐ Version 2 smoke tests and critical workflows passed.
- ☐ Rollout expansion explicitly approved.
- ☐ Production monitoring and customer signals remained stable.
- ☐ Final evidence and lessons added to the runbook.

# Appendix C. Glossary

| Term | Meaning |
|---|---|
| Backward compatibility | Ability of an older application version to operate against the changed database state. |
| Database dump | A captured representation of database structure and/or data at a specific point. |
| Expand-migrate-contract | Migration pattern that adds compatible structures, backfills/cuts over, then removes deprecated structures. |
| Idempotent migration | Migration that can be rerun without duplicating or corrupting business effects. |
| Reconciliation | Comparison of source, target, and expected business state after transformation. |
| Rollback | Return to a prior known-good application and/or database state. |
| Forward correction | A new change that repairs the current state without restoring the previous version. |
| Phased rollout | Controlled expansion of a release across progressively larger scopes. |
| Release invariant | A condition that must remain true at every point in the deployment sequence. |

**Document boundary**

The historical facts in this case study are limited to the Version 2 production migration, phased rollout, concern raised about correctness, use of database dumps, collaboration with the database team, timestamp confusion, and the evidence-based conclusion. The detailed controls and modernization patterns provide a defensible engineering framework around those facts without asserting undocumented implementation details.